

Version: 1.00



Selection

Bienvenue !

Résumé

Bienvenue à 42 ! Cette activité vous introduira en douceur aux compétences et connaissances de base nécessaires pour bien commencer.

#Shell

#Unix

#Git

42

Avertissement relatif à la propriété intellectuelle

L'ensemble du contenu présenté dans ce module de formation — y compris, sans s'y limiter, les textes, images, graphiques et autres éléments — est protégé par les droits de propriété intellectuelle détenus par l'Association 42.

Conditions d'utilisation

- **Usage personnel** : L'utilisation du contenu de ce module est strictement réservée à un usage personnel. Toute reproduction, diffusion, modification, utilisation publique ou commerciale est interdite sans l'autorisation écrite préalable de l'Association 42.
- **Respect de l'intégrité** : Il est interdit de modifier, altérer ou adapter le contenu de manière à en dénaturer le sens ou à porter atteinte à son intégrité.

Protection des droits

Toute violation des présentes conditions constitue une atteinte aux droits de propriété intellectuelle et peut faire l'objet de poursuites. L'Association 42 se réserve le droit d'engager toute action nécessaire à la protection de ses droits, notamment par voie judiciaire et en réclamant des dommages et intérêts le cas échéant.

Pour toute question relative à l'utilisation du contenu ou pour obtenir une autorisation, veuillez contacter : legal@42.fr

Table des matières

1	Instructions	1
2	Préambule	5
3	Tutoriel : L'arbre de la vie	6
4	Tutoriel : Dansons avec le Shell ?	7
5	Tutoriel : Utilisons GIT !	10
6	Exercice 0 : Z	14
7	Exercice 1 : SSH moi !	15
8	Soumission et évaluation par les pairs	16

Chapitre 1

Instructions

- Seule cette page servira de référence : ne faites pas confiance aux rumeurs.
- Attention ! Ce document pourrait potentiellement changer jusqu'à la soumission.
- Assurez-vous d'avoir les permissions appropriées sur vos fichiers et répertoires.
- Vous devez suivre les procédures de soumission pour tous vos exercices.
- Vos exercices seront vérifiés et notés par vos camarades de classe.
- De plus, vos exercices seront vérifiés et notés par un programme appelé Moulinette.
- Moulinette est très méticuleuse et stricte dans son évaluation de votre travail. Elle est entièrement automatisée, et il n'y a aucun moyen de négocier avec elle. Donc, pour éviter les mauvaises surprises, soyez aussi minutieux que possible.
- Moulinette n'est pas très ouverte d'esprit. Elle n'essayera pas de comprendre votre code s'il ne respecte pas la Norme. Moulinette s'appuie sur un programme appelé `norminette` pour vérifier si vos fichiers respectent la norme. En bref : il serait stupide de soumettre un travail qui ne passe pas la vérification de `norminette`.
- Ces exercices sont soigneusement disposés par ordre de difficulté - du plus facile au plus difficile. Nous ne considérerons pas un exercice plus difficile réussi si un exercice plus facile n'est pas parfaitement fonctionnel.
- Utiliser une fonction interdite est considéré comme de la tricherie. Les tricheurs obtiennent -42, et cette note est non négociable.
- Vous n'aurez à soumettre une fonction `main()` que si nous demandons un programme.
- Moulinette compile avec ces flags : `-Wall -Wextra -Werror`, et utilise `cc`.
- Si votre programme ne compile pas, vous obtiendrez 0.
- Vous ne pouvez pas laisser aucun fichier supplémentaire dans votre répertoire autre que ceux spécifiés dans le sujet.
- Vous avez une question ? Demandez à votre pair à droite. Sinon, essayez votre pair à gauche.
- Votre guide de référence s'appelle `Google / man / Internet / ....`
- Consultez le Slack Piscine.
- Examinez attentivement les exemples. Ils pourraient très bien appeler à des détails qui ne sont pas explicitement mentionnés dans le sujet...

- Par Odin, par Thor! Utilisez votre cerveau!!!



N'oubliez pas d'ajouter l'*en-tête standard 42* dans chacun de vos fichiers .c/.h. Norminette vérifie son existence de toute façon!



Norminette doit être lancée avec le flag `-R CheckForbiddenSourceHeader`. Moulinette l'utilisera aussi.

● Contexte

La Piscine C est intense. C'est votre premier grand défi à 42 — une plongée profonde dans la résolution de problèmes, l'autonomie et la communauté.

Durant cette phase, votre objectif principal est de construire vos fondations — à travers la lutte, la répétition, et surtout l'échange d'**apprentissage par les pairs**.

À l'ère de l'IA, les raccourcis sont faciles à trouver. Cependant, il est important de considérer si votre utilisation de l'IA vous aide vraiment à grandir — ou si elle se contente de vous empêcher de développer de vraies compétences.

La Piscine est aussi une expérience humaine — et pour l'instant, rien ne peut remplacer cela. Pas même l'IA.

Pour un aperçu plus complet de notre position sur l'IA — en tant qu'outil d'apprentissage, dans le cadre du programme ICT, et en tant qu'attente croissante sur le marché du travail — veuillez vous référer à la FAQ dédiée disponible sur l'intranet.

● Message principal

- 👉 Construire des fondations solides sans raccourcis.
- 👉 Développer réellement des compétences techniques et personnelles.
- 👉 Expérimenter un véritable apprentissage par les pairs, commencer à apprendre à apprendre et à résoudre de nouveaux problèmes.
- 👉 Le parcours d'apprentissage est plus important que le résultat.
- 👉 Apprendre les risques associés à l'IA, et développer des pratiques de contrôle efficaces et des contre-mesures pour éviter les pièges courants.

● Règles pour les apprenants :

- Vous devez appliquer le raisonnement à vos tâches assignées, surtout avant de vous tourner vers l'IA.
- Vous ne devez pas demander de réponses directes à l'IA.
- Vous devez apprendre l'approche globale de 42 sur l'IA.

● Résultats de la phase :

Dans cette phase de fondation, vous obtiendrez les résultats suivants :

- Acquérir des bases techniques et de codage appropriées.
- Savoir pourquoi et comment l'IA peut être dangereuse pendant cette phase.

● Commentaires et exemple :

- Oui, nous savons que l'IA existe — et oui, elle peut résoudre vos activités. Mais vous êtes ici pour apprendre, pas pour prouver que l'IA a appris. Ne gaspillez pas votre temps (ou le nôtre) juste pour démontrer que l'IA peut résoudre le problème donné.
- Apprendre à 42 ne consiste pas à connaître la réponse — il s'agit de développer la capacité à en trouver une. L'IA vous donne la réponse directement, mais cela vous empêche de construire votre propre raisonnement. Et le raisonnement prend du temps, des efforts, et implique des échecs. Le chemin vers le succès n'est pas censé être facile.
- Gardez à l'esprit que lors des examens, l'IA n'est pas disponible — pas d'internet, pas de smartphones, etc. Vous réaliserez rapidement si vous avez trop compté sur l'IA dans votre processus d'apprentissage.
- L'apprentissage par les pairs vous expose à différentes idées et approches, améliorant vos compétences interpersonnelles et votre capacité à penser de manière divergente. C'est bien plus précieux que de simplement discuter avec un bot. Alors ne soyez pas timide — parlez, posez des questions, et apprenez ensemble !
- Oui, l'IA fera partie du programme — à la fois comme outil d'apprentissage et comme sujet en soi. Vous aurez même l'occasion de construire votre propre logiciel d'IA. Afin d'en apprendre davantage sur notre approche crescendo, vous passerez par la documentation disponible sur l'intranet.

✓ Bonne pratique :

Je suis bloqué sur un nouveau concept. Je demande à quelqu'un à proximité comment il l'a abordé. Nous parlons pendant 10 minutes — et soudain, ça fait clic. Je comprends.

✗ Mauvaise pratique :

J'utilise secrètement l'IA, je copie du code qui semble correct. Pendant l'évaluation par les pairs, je ne peux rien expliquer. J'échoue. Pendant l'examen — sans IA — je suis à nouveau bloqué. J'échoue.

Chapitre 2

Préambule

Si vous ne l'avez pas encore fait, c'est le moment idéal pour vous présenter aux personnes assises à côté de vous !

Chapitre 3

Tutoriel : L'arbre de la vie

Bienvenue ! Avant de plonger directement dans le sujet, savez-vous comment naviguer parmi les fichiers et les dossiers présents sur votre système ? Peut-être savez-vous déjà ce qu'est un fichier et un répertoire, mais attendez ! Il y a plus ! Avec la personne à côté de vous, recherchez ce qu'est l'**arbre des répertoires UNIX**. Identifiez et comprenez les concepts de **fichier**, **répertoire**, **structure arborescente**, **racine**.

Une fois que vous vous sentez tous les deux à l'aise avec ces notions, tournez-vous vers une autre personne à côté de vous et vérifiez votre compréhension avec elle.

Chapitre 4

Tutoriel : Dansons avec le Shell ?

- Découvrez comment ouvrir un terminal. Si vous vous retrouvez face à une fenêtre noire qui semble attendre que vous y tapiez quelque chose, vous êtes sur la bonne voie.
- Lorsque vous ouvrez votre **terminal**, il lance automatiquement un programme qui deviendra votre meilleur ami à partir de maintenant : le **shell**. C'est ce shell qui lira les **commandes** que vous tapez à l'**invite de commande** et les exécutera. Si aucun programme ne s'exécute plus dans votre terminal, la fenêtre se ferme. Trouvez la commande pour quitter le shell et fermer le terminal.



Cliquer sur la petite croix rouge dans la fenêtre n'est pas la solution que vous cherchez.

- Ouvrez un nouveau terminal, puis trouvez la commande qui affiche le nom du **répertoire de travail actuel**. Ce répertoire est votre répertoire personnel. Il vous est propre et est le répertoire par défaut lorsque vous ouvrez un terminal.
- Les commandes peuvent également accepter des **options de commande** pour contrôler et personnaliser leur comportement. En particulier, toutes les commandes disponibles acceptent une option pour afficher un message d'aide. Trouvez comment afficher le message d'aide de la commande qui affiche le répertoire de travail actuel.



Vous ne savez pas par où commencer ? Demandez à la personne à côté de vous de chercher cela ensemble.

- Votre shell peut faire bien plus que simplement afficher le répertoire de travail actuel. Par exemple, il peut également **lister** le contenu du répertoire de travail actuel ! Trouvez la commande qui liste le contenu du répertoire de travail actuel et essayez-la.
- Cette commande est un bon exemple de commande qui peut accepter de nombreuses options pour contrôler son comportement. En particulier, cette commande a de nombreuses options utilisées pour contrôler le formatage de sa sortie. Trouvez l'option qui formate la

sortie sous forme de **liste longue**.

- Maintenant que vous savez que `ls` accepte des options, il est temps de découvrir une fonctionnalité intéressante des systèmes de fichiers UNIX : les fichiers et répertoires cachés. Pas de panique, il n'y a pas de piratage impliqué (pour l'instant...), cette fonctionnalité est simplement utilisée pour garder certains fichiers moins intéressants hors de la liste par défaut. Cependant, il est possible de les afficher quand même. Trouvez un moyen d'afficher les fichiers et dossiers cachés dans votre répertoire de travail actuel.



Vous devriez vérifier avec la personne à côté de vous si elle sait ce qui rend un fichier ou un dossier caché.

- Trouvez la commande pour **changer de répertoire**. Naviguez jusqu'au répertoire racine (`/`), puis **listez** son contenu. Reconnaissez-vous certains répertoires de votre chemin personnel ?
- Naviguez jusqu'à un répertoire appelé `Utilisateurs` (ou similaire), puis **listez** son contenu. Reconnaissez-vous certains noms ?
- Minimisez le terminal et, dans votre dossier `Documents`, faites un clic droit et sélectionnez "Nouveau dossier", nommez-le `I_love_42`. Ensuite, retournez dans votre terminal et changez le répertoire vers votre dossier `Documents`, qui se trouve dans votre **dossier personnel**. Une fois dedans, vérifiez si le dossier que vous venez de créer est présent.
- Créez un autre dossier dans votre répertoire `Documents`, mais cette fois, vous devez **créer le dossier** en utilisant votre terminal. Nommez-le : `I_adore_42_even_more_now`.
- Vérifiez que le dossier est bien apparu dans votre répertoire `Documents`. Que s'est-il passé ?
- Créez un fichier nommé `HelloWorld.txt` dans votre répertoire. Écrivez-y du texte en utilisant un éditeur de texte disponible dans votre terminal. Après l'avoir fait, listez en format long ce qui se trouve dans votre dossier. Demandez à votre voisin de droite ce que signifie le code `"-rw-r-r-"` qui apparaît au début de la ligne.
- Si leur explication était correcte, vous devriez avoir appris sur les **droits d'accès UNIX** et que vous êtes maintenant capable de modifier le fichier que vous venez de créer. Écrivez-y 'Hello World'. Il existe de nombreuses façons de modifier des fichiers ; prenez le temps de découvrir laquelle sera la plus pratique à l'avenir.



Indice : ce n'est pas **echo**. Mais certains **éditeurs de texte** sont disponibles dans votre terminal.

- Affichez le contenu de votre fichier en utilisant la commande appropriée.
- Trouvez la commande qui vous permet de **supprimer un répertoire** et supprimez celui (ou ceux) que vous venez de créer.



Dans la vie d'un programmeur, lire le **manuel** devrait devenir un réflexe ! Cherchez ce qu'est un manuel, puis lisez le manuel de la commande **man**. Faites toujours `man <command>` avant d'utiliser une commande pour la première fois.

Chapitre 5

Tutoriel : Utilisons GIT !



Avant de commencer, assurez-vous d'avoir terminé le tutoriel sur le shell et d'être à l'aise avec les commandes de base du terminal.

- **Git** est un système de contrôle de version qui stockera votre travail pour chaque projet. Recherchez ce qu'est un **dépôt**, un **système de contrôle de version**, et comprenez le concept de base de **Git**. Discutez avec votre voisin : pourquoi les programmeurs pourraient-ils avoir besoin d'enregistrer différentes versions de leur code ?



Pensez à l'écriture d'un document - ne voudriez-vous pas garder les versions précédentes au cas où vous feriez une erreur ?

- **Git** est comme une machine à remonter le temps pour votre code. Il enregistre des instantanés de votre travail appelés **commits**. Un **dépôt** est comme un dossier de projet que **Git** surveille pour les modifications.
- Chacun de vos projets vous donne un dépôt dédié sur un serveur **Github**. Trouvez l'URL du dépôt de votre projet de tutoriel sur l'intranet et copiez-la.
- La plupart des commandes git peuvent être appelées en utilisant la syntaxe suivante `git <command> <arguments>`. Découvrez quelle est la commande pour **cloner** votre dépôt.
- Essayez la commande suivante :

Terminal :

```
?> git clone <votre_url_de_dépôt>
```

La commande devrait échouer. Savez-vous pourquoi ?

- Vos dépôts sont stockés sur des serveurs **Github**. Vérifiez avec vos pairs si vous connaissez d'autres plateformes de contrôle de version pour Git.
- Vos dépôts sont stockés sur un serveur externe. Pour accéder depuis votre poste de travail à vos dépôts, vous devez d'abord comprendre ce qu'est une **clé SSH**.
- Une fois que vous avez fait quelques recherches sur ce qu'est une clé SSH, vous devrez découvrir comment l'ajouter à votre compte Github. Certains de vos pairs devraient déjà

savoir comment faire.



Avant d'ajouter une clé, vous devrez également trouver un moyen de **générer** une nouvelle **clé SSH**.

- Quel est le nom de la commande pour générer une nouvelle paire de clés SSH ?
- Une fois que vous avez généré une nouvelle paire de clés SSH, affichez votre clé publique (la partie que vous pouvez partager) avec :

Terminal :

```
?> cat ~/.ssh/id_ed25519.pub
```

Cette sortie est la partie que vous devez stocker dans d'autres services (comme les serveurs de contrôle de version). Cela se fait généralement dans la section Paramètres. Cela indique au service de "faire confiance à cet ordinateur".



Rappelez-vous : les clés SSH fonctionnent par paires : gardez votre clé privée secrète, partagez votre clé publique pour accorder l'accès. Vous devez être sûr d'avoir accompli cette étape avant de passer aux parties suivantes. Vous ne pourrez pas valider un exercice sans cela.

- Une fois que vous êtes sûr d'avoir ajouté votre clé SSH à votre compte Github, essayez à nouveau de **cloner** votre dépôt pour ce projet.

Terminal :

```
?> git clone <votre_url_de_dépôt>
```

- Naviguez dans le dossier créé. Quel est le nom du dossier ? Listez son contenu, y compris les fichiers cachés.
- Vous devriez voir un dossier caché **.git** - c'est là que Git stocke toute sa magie ! Vérifiez ce que Git pense de votre dépôt :

Terminal :

```
?> git status
```

Cette commande est utilisée pour voir la différence entre vos modifications locales et le dépôt sur Github.

- Créez un fichier de test simple. Par exemple :

Terminal :

```
?> echo "Hello Git!" > test.txt
```

- Exécutez la commande qui vous permet de vérifier le **statut** de votre dépôt. Qu'est-ce qui a changé ?
- Lisez le message attentivement. Expliquez à l'un de vos camarades ce que signifie **fichier non suivi**.
- Exécutez la commande pour **suivre** votre fichier et vérifiez le **statut** à nouveau. Qu'est-ce qui a changé ?



Suivre un fichier est simplement l'action de dire à Git d'**ajouter** le fichier et ses modifications locales actuelles, qu'elles sont prêtes à être enregistrées.

- Ce que vous venez de faire est de changer le **statut** de votre fichier de **non suivi** à **prêt à être validé** : Git a enregistré une copie locale du fichier dans son état actuel et est maintenant prêt à suivre les modifications. Vérifions cela ! Modifiez votre fichier et vérifiez à nouveau le **statut**.
- Vous pouvez voir comment git vous indique que vous avez modifié votre fichier et vous donne deux options : vous pouvez soit enregistrer les modifications en exécutant la commande que vous avez utilisée auparavant, soit ignorer vos modifications locales. Faisons le second, vérifiez ce qui est arrivé à vos modifications et vérifiez le statut à nouveau.
- Lorsque vous ajoutez un fichier, git enregistre son état localement, mais le serveur ne sait pas encore que vous avez créé un fichier. Vous devez étiqueter vos modifications, puis les envoyer au serveur. Cherchez un moyen de **valider** (enregistrer) vos modifications avec un message descriptif.



Nous vous recommandons d'être explicite avec le message, en décrivant le contenu que vous validez.

- Une fois que vous avez utilisé la commande pour **valider**, vérifiez à nouveau le statut de vos fichiers et vérifiez comment il a changé. Maintenant, trouvez la commande qui vous permet de vérifier la liste de vos **validations**.



Git **enregistre** tout ce que vous **validez** sur le serveur.

Savez-vous ce que signifie la partie (HEAD -> main) ?

- Si vous trouvez dans l'historique la présence de votre validation, cela signifie que vos modifications sont enregistrées localement. Mais le serveur ne le sait pas encore. Cherchez la commande pour **pousser** votre validation vers le serveur.

- Par défaut, lorsque vous clonez un dépôt, vous serez sur la branche `main`, et puisque nous ne l'avons pas changée, vous y êtes toujours. Exécutez la commande pour pousser vos validations vers le serveur. (Pour votre information : vous êtes sur le **remote** "origin").
- Vérifiez à nouveau la liste des validations, qu'est-ce qui a changé ? Remarquez la date, l'auteur, et votre commentaire.
- Pour vérifier que cela a fonctionné, allez dans votre répertoire personnel et **clonez** votre dépôt à nouveau dans un dossier différent. Votre fichier devrait être là ! Vous pouvez également vérifier les validations, des surprises ?
- Enfin, mais non des moindres, rappelez-vous que votre dépôt ne doit pas contenir de fichiers ou dossiers inutiles. Supprimez donc votre fichier de votre répertoire local, et procédez à l'enregistrement de vos modifications :

Terminal :

```
?> rm test.txt
?> git add test.txt
?> git commit -m "Supprimer le fichier de test"
?> git push origin main
```

- Une fois que vous avez pratiqué ce flux de travail de base de Git plusieurs fois, vous devriez être prêt à commencer à travailler sur vos exercices ! N'oubliez pas : votre dépôt ne doit contenir que les fichiers nécessaires à votre projet. Et n'hésitez pas à valider votre compréhension avec vos pairs. Amusez-vous bien !

Chapitre 6

Exercice 0 : Z

	Exercice: 0	
Z		
Répertoire: ex0/		
Fichiers à Rendre: z		
Autorisé: None		

- Créez un fichier appelé z qui retourne "Z", suivi d'une nouvelle ligne, chaque fois que la commande cat est utilisée sur celui-ci.

Sortie du Terminal

```
?> cat z
Z
?>
```



Vous ne savez pas par où commencer et ce qu'est un terminal ? Demandez aux personnes autour de vous et trouvez ensemble !

Chapitre 7

Exercice 1 : SSH moi !

	Exercice: 1	
Clé SSH		
Répertoire: ex1/		
Fichiers à Rendre: id_ed25519.pub		
Autorisé: None		

Créez votre propre clé SSH. Une fois cela fait :

- Ajoutez votre clé publique à votre dépôt, dans un fichier nommé id_ed25519.pub.



- Le nom du fichier n'a pas été choisi au hasard.
- Assurez-vous de comprendre la différence entre la clé publique et la clé privée.



Avez-vous vérifié avec vos voisins ?

Chapitre 8

Soumission et évaluation par les pairs

Soumettez votre devoir dans votre dépôt Git comme d'habitude. Seul le travail dans votre dépôt sera évalué lors de la défense. Assurez-vous de vérifier deux fois les noms de vos fichiers pour vous assurer qu'ils sont corrects.



Vous devez soumettre uniquement les fichiers demandés par le sujet de ce projet.